Rayquaza — A Plain-English Primer

Ojas Mutreja, Vedanth Dama

Defence Research and Development Organisation — Scientific Analysis Group (DRDO SAG)

A companion to the technical paper. This is the on-ramp: it starts from zero — no cryptography, no AI background assumed — and ramps steadily up to the point where the paper takes over. If you can read a newspaper, you can read this. When you reach the end, the paper's abstract will make sense.

For the story-driven, per-leak walkthrough using a single running analogy, see the companion Bank Drop-Box briefing. This primer is the wider bridge: why the problem exists, what each technology is, how they are wired together, and what we found.

Part 1 Why this research exists

The quantum threat, briefly

Almost everything private online today — messages, banking, logins — is protected by encryption that relies on one bet: that certain mathematics is too hard for any computer to undo in a reasonable time. Today's schemes (RSA, elliptic-curve) rest on problems like factoring enormous numbers. Ordinary computers cannot do this fast enough to matter.

A large **quantum computer** would change that. There is a known quantum algorithm (Shor's) that factors those numbers efficiently — which would break RSA and elliptic-curve encryption outright. Large enough quantum computers do not exist yet, but the threat is already here in one specific way: an adversary can **record encrypted traffic today and decrypt it years later** once the hardware arrives. Security agencies call this "harvest now, decrypt later." Anything that must stay secret for a decade is already at risk.

Post-quantum cryptography (PQC)

The response is **post-quantum cryptography** — new encryption built on different mathematics that is believed to resist *both* ordinary and quantum computers. In 2024, the U.S. standards body NIST finalised the first winners: **ML-KEM (formerly CRYSTALS-Kyber)** for key exchange, and **ML-DSA (formerly CRYSTALS-Dilithium)**

for digital signatures. Governments and industry worldwide — DRDO included — are now migrating to them.

The catch this project is about

Here is the crucial point, and the reason Rayquaza exists: **the maths being unbreakable does not mean the software is safe.** A cryptographic scheme is a mathematical design; a real system is thousands of lines of hand-written, performance-tuned C code that *implements* that design. The lock can be perfect while the person operating it gives the secret away through a careless habit.

The most important such habit is a **timing side-channel** — when the code takes a slightly different amount of time depending on the secret it is handling. An attacker who cannot break the maths can instead stand outside with a metaphorical stopwatch, measure those tiny time differences thousands of times, and reconstruct the secret key. This has already broken real PQC deployments (the “KyberSlash” bug, 2023).

Rayquaza studies this gap: not the maths (nobody here tries to break the lock), but whether an AI can *read the code* and find these timing mistakes the way an attacker eventually would — faster and at scale.

Part 2 The technologies, one at a time (simple → technical)

A key-exchange mechanism (ML-KEM / Kyber). *Simple:* a drop-box on a bank wall — anyone can post a sealed package, only the manager’s private key can open it, and opening it produces a shared secret both sides then use to talk privately. *Technical:* a Key Encapsulation Mechanism (KEM) with three operations — `keypair`, `encapsulate`, `decapsulate`. Decapsulation runs the **Fujisaki–Okamoto (FO) transform**, which re-encrypts the recovered message and compares it to the received ciphertext to reject tampering. That comparison step is a classic leak site.

A signature scheme (ML-DSA / Dilithium). *Simple:* a wax seal the bank stamps on outgoing letters so recipients can verify they are genuine and unforged. *Technical:* verification recomputes a challenge value and compares it to the one in the signature — another comparison step, another potential leak site.

A timing side-channel. *Simple:* if the manager takes longer on a genuine package than a forged one, a patient observer with a stopwatch learns the secret without touching the lock. *Technical:* execution time depends on secret data. It is measured with a two-sample statistical test (**Welch's t-test**) over tens of thousands of runs; a large **t-statistic** means the time difference is real, not measurement noise. A t of 2–3 is the usual bar for "real"; every leak in this study clears it by 25× to 1000×.

Constant-time code. *Simple:* the manager walks through every step at exactly the same pace regardless of what's inside. *Technical:* every branch taken, memory address touched, and comparison made is independent of secret data. Breaking this — even by one `if` on a secret, or one ordinary `memcmp` instead of its constant-time cousin — creates a measurable channel.

The two mistake families we tested. Nearly all timing leaks reduce to two shapes: - **A secret-dependent branch** — an `if` / `for` / `while` whose condition depends on the secret, so the code physically takes a different path (and time) for different secrets. - **A non-constant-time comparison** — using a standard `memcmp` that *stops early* at the first mismatch, so the time reveals *where* two values first differ. (This is the KyberSlash family.)

Part 3 What we actually built (the pipeline)

The idea in one line: **hand the code's blueprint to an AI, ask "where might the operator behave inconsistently?", then confirm each guess with a real stopwatch.** It runs as a closed loop of four steps:

1. **Read** — an AI reads the C source and produces ranked guesses ("this `if` on a secret coefficient looks like a leak, here, line 25"). A fast built-in text scan also flags known leak-shaped patterns (like a bare `memcmp`) to steer the AI when it overlooks one.
2. **Write a test** — an AI writes a small timing experiment (a "harness") that runs the suspect function two ways: an input that should trigger the leak, and one that should not.
3. **Measure** — a plain C stopwatch program runs that experiment 50,000 times and applies Welch's t-test. A big t means a real leak; a small t means no signal. This is the **ground-truth oracle** — the maths, not an opinion.
4. **Judge** — a second AI reads the stopwatch result and decides whether the original guess held up (promote / demote / reject), and if confirmed, sketches how an attacker would exploit it. Its verdict feeds back into step 1 for the next round.

Two facts that matter for the paper's argument:

- **The core pipeline runs on open-weight models, locally, with no internet.** The reader/writer is `codellama:7b` and the judge is `qwen3:8b`, both served on the machine itself. Nothing is sent to an external cloud — which is exactly what makes the method usable in classified, air-gapped security work. *(Part 4.5 below covers a separate follow-up study that deliberately swaps in paid cloud AI models — including Claude — to answer a specific question; the original, air-gapped-capable pipeline described here does not depend on them.)*
- **We compared against the traditional tool.** Alongside the AI, we ran **AFL++** — the standard automated bug-finder ("fuzzer") — for 24 hours on the same targets, as a fair baseline.

To test all this without endangering anything real, we took correct reference implementations and **deliberately planted** small, realistic mistakes — six of them, five in Kyber and one in Dilithium — then checked whether the AI could rediscover them from the source alone, with no hint about what we had planted.

Part 4 What we found (plain, then where to get the numbers)

- **The AI found 4 of the 5 Kyber leaks entirely on its own** — every one of the "secret-dependent branch" kind — pinpointing the exact function and line. The 5th (the `memcmp` comparison leak) it missed unaided, because spotting it requires knowing that a function *should* have used the safe comparison — a subtler kind of reasoning. A one-line nudge from the text scan fixed it.
- **The traditional fuzzer (AFL++) found nothing — zero — in 24 hours** (~120 million runs per target). And this is the striking part: it is not that the fuzzer was *too slow*; it is **blind by design.** A fuzzer maps which code *paths* run. The `memcmp` leak runs the **same path every time** — only the *timing* changes — so the fuzzer's map of the leaky code was **byte-for-byte identical to the safe code.** Timing lives in a dimension the fuzzer cannot see.
- **A hardware twist:** the same `memcmp` leak is clearly measurable on x86 laptop/server chips but **invisible on ARM chips** at standard optimisation, because ARM compiles the short comparison into a fixed-width instruction with no early exit. An audit run only on ARM would miss a leak that is real on x86 — a genuinely useful practitioner warning.

The exact statistics, the per-leak breakdown, and four figures visualising all of this are in the technical paper (see especially Figures 1–4 and Tables 2–3).

Part 4.5 Then we asked: does paying for a bigger AI fix the blind spot?

The one gap in Part 4's story is the 5th leak — the AI needed a one-line hint to catch the `memcmp`-style mistake. The obvious follow-up question: is that just because we used a small, free, local AI? Would a much bigger, much more expensive model — the kind you pay for by the API call — simply *know better* and catch it unaided?

We tested this directly, rather than guessing. Over one (very long) night we ran the same pipeline against **18 different AI models** — everything from the original free

local model, up through mid-size models on rented cloud graphics cards, up to the most capable paid models from both Anthropic (Claude) and OpenAI (GPT), including their most expensive "flagship" tiers. Every model got the same four test cases, with no cheating: same source code, same rules, same scorekeeping.

The honest answer is no — paying more does not fix it.

- Every single model, cheap or expensive, local or cloud, caught **100% of the "secret-dependent branch" leaks** — the kind Part 4 said was easy. Zero misses, zero exceptions, across all 18 models. That part of the story holds up completely.
- On the harder `memcmp`-style leaks, though, accuracy dropped to **64% overall** — and it did **not** improve with a bigger price tag. The single most expensive model in the whole study missed one of the two hard cases. The single *cheapest* paid model tied for the best score of any model tested, local or paid, at a cost of about one-and-a-half cents for all four test cases combined.
- We even found a model — Anthropic's most capable one — that flatly **refused to look at the code at all**, flagging it as sensitive "cyber" content, even though every other model (16 separate attempts) analysed the exact same kind of function without any objection. The most capable AI in the study was, in this one specific case, the *least* usable one.

The takeaway for anyone building a tool like this: don't assume the priciest AI model is automatically the best choice. For this particular job — reading real cryptographic code and spotting a specific kind of subtle bug — a cheap or even free model did just as well, and sometimes better, than models costing far more per use. The full breakdown, including which models scored what and why, is in the paper's new Section 5 and its accompanying figures.

One more test, to make sure the answer wasn't just an artefact of how we asked. So far, every model — cheap or expensive — got a one-line nudge telling it to double-check for this specific kind of mistake. What if the nudge, not the model's own judgement, was doing all the work? We removed the nudge entirely and reran five of

the models — one small free one, and four of the priciest cloud ones — asking them to find the hardest bug completely unassisted.

They all missed it. Every single one — including three models that had found the exact same bug correctly moments earlier, when given the one-line hint. Take the hint away, and even the most expensive AI available reverts to missing it. This settles the question cleanly: the nudge isn't a crutch for a weak model that a stronger model would outgrow — it is doing real, necessary work that no amount of extra AI horsepower currently replaces. (There was one silver lining: on an *easier* version of the same kind of bug, the expensive cloud models did manage to find it unassisted, while the small free model still needed the hint — so bigger models are not entirely unaffected by scale, just not enough to close the gap on the hardest case.)

Part 5 Plain-word ↔ paper-word bridge

Read this once and the paper's vocabulary will feel familiar:

Plain term (this primer / the analogy)	Paper / technical term
The drop-box	Key Encapsulation Mechanism (KEM) — ML-KEM / Kyber512
The wax seal	Digital signature scheme — ML-DSA / Dilithium (ML-DSA-44)
Re-seal-and-compare step	Fujisaki–Okamoto (FO) transform comparison
Stopwatch attack	Timing side-channel
“Give up at first mismatch” comparison	Non-constant-time `memcmp` (`nonconstant_comparison` class)
“Pause to decide” on a secret	Secret-dependent branch (`secret_dependent_branch` class)
Walks every step at the same pace	Constant-time implementation
Confidence number	Welch t-statistic
The stopwatch program	The timing oracle (Welch t-test, n=50,000)
Traditional brute-force bug-finder	AFL++ coverage-guided fuzzer
The AI reader / test-writer	`codellama:7b` (Stage 1 / Stage 3)
The AI judge	`qwen3:8b` (Stage 2 refinement)
Deliberately planted mistake	Weakened target / planted vulnerability
The AI could reach any of 18 models through one connector	Model Gateway (`sandbox/gateway` , Router + Meter)
Got the right answer	`located` (correct category + location)
The stopwatch found a real signal (not necessarily the right answer)	`confirmed` (oracle-significant; independent of whether the model was correct)

You are now ready for the paper. It says the same things, with the full rigour, the exact numbers, and the related research that surrounds this work.